MODUL PRAKTIKUM | PEMROGRAMAN INTERNET

API Pencatatan Pengukuran Sensor

Manajemen Basis Data Relasional dan ORM dengan Python

Fokus CPMK-4
Merancang entitas basis data relasional dan melakukan operasi CRUD menggunakan ORM. Implementasi praktikum ini menggunakan FastAPI, SQLModel, dan SQLite.

Untuk mahasiswa Teknik Elektro

Estimasi: 2 sesi praktikum x 100 menit | Edisi 21 Oktober 2026

1. Tujuan dan Petunjuk Praktikum

Praktikum ini membawa konsep tabel, relasi, dan CRUD ke kasus yang akrab di laboratorium elektro: pencatatan sensor dan nilai pengukurannya.

Capaian pembelajaran

- Membuat dua entitas relasional: Sensor dan Pengukuran.
- Menjelaskan primary key, foreign key, dan relasi satu-ke-banyak.
- Membuat tabel dan menyimpan objek Python ke SQLite melalui ORM.
- Menyediakan endpoint REST untuk Create, Read, Update, dan Delete.
- Menguji request JSON serta memeriksa respons sukses dan kesalahan umum.

Prasyarat

Mahasiswa sudah mengenal variabel, fungsi, class sederhana, dictionary/JSON, HTTP GET dan POST, serta konsep dasar tabel dan kunci. Tidak diperlukan pengalaman memakai Java atau Spring.

Peralatan

Perangkat lunak	Fungsi
Python 3.10 atau lebih baru	Menjalankan aplikasi dan paket praktikum.
Terminal PowerShell	Membuat lingkungan virtual, memasang paket, dan menjalankan server.
Browser	Membuka dokumentasi interaktif API di /docs.

Cara memakai modul
Proyek sudah dibagi per fungsi. Buka file sesuai langkah dan bandingkan dengan potongan kode. **main.py** hanya merakit aplikasi; model, sesi database, CRUD, dan endpoint berada di modul masing-masing.

2. Kasus dan Rancangan Data

Sebuah laboratorium memiliki beberapa sensor. Setiap sensor menghasilkan banyak catatan pengukuran dari waktu ke waktu. Contoh parameter: tegangan, arus, atau suhu.

Kamus data

Tabel	Kolom	Makna
Sensor	id (PK)	Nomor unik sensor yang dibuat oleh basis data.
Sensor	code, location, parameter	Kode sensor, lokasi pemasangan, dan besaran yang diukur.
Pengukuran	id (PK)	Nomor unik satu catatan pengukuran.
Pengukuran	sensor_id (FK)	Sensor yang menghasilkan catatan tersebut.
Pengukuran	value, unit, measured_at	Nilai, satuan, dan waktu pengukuran.

Struktur berkas modular

Berkas	Tanggung jawab
main.py	Membuat aplikasi FastAPI dan memasang router.
database.py	Engine SQLite, sesi database, dan pembuatan tabel.
models.py / schemas.py	Model tabel ORM dan bentuk request/response.
crud.py	Fungsi akses data Create, Read, Update, dan Delete.
routers/sensors.py	Endpoint sensor dan daftar pengukuran per sensor.
routers/measurements.py	Endpoint CRUD untuk pengukuran.
requirements.txt	Daftar paket Python yang diperlukan.

3. Persiapan Proyek di Windows

Buka PowerShell di folder proyek. File proyek praktikum ini berada di C:\xampp\htdocs\praktikum_sensor. Buat lingkungan virtual dan pasang paket dari requirements.txt.

```
cd C:\xampp\htdocs\praktikum_sensor
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Jika perintah **py** tidak tersedia, ganti dengan **python**. Folder **htdocs** di sini hanya menjadi lokasi proyek. Server FastAPI dijalankan oleh Uvicorn, bukan oleh Apache/PHP XAMPP.

Lingkungan virtual
Perintah di modul selalu memakai interpreter di **.venv**. Dengan begitu mahasiswa tidak perlu mengaktifkan virtual environment terlebih dahulu.

Paket yang digunakan

Paket	Peran
FastAPI	Menerima request HTTP dan mengembalikan respons JSON.
SQLModel	Mendefinisikan model ORM dan berinteraksi dengan SQLite.
Uvicorn	Menjalankan server aplikasi FastAPI di komputer lokal.
SQLite	Basis data satu file; tidak memerlukan layanan database terpisah.

Jangan memasang FastAPI atau Uvicorn secara global untuk praktikum ini. Jalankan pip melalui file python.exe di dalam .venv seperti contoh di atas.

4. Langkah 1 - Model ORM dan Schema API

Buka **models.py**. Class Sensor dan Measurement menjadi tabel. Field sensor_id adalah foreign key; Relationship menyatakan bahwa satu sensor dapat memiliki banyak pengukuran.

```
from sqlmodel import Field, Relationship, SQLModel


class Sensor(SQLModel, table=True):
    __tablename__ = "sensor"
    id: int | None = Field(default=None, primary_key=True)
    code: str = Field(index=True)
    location: str
    parameter: str
    measurements: list["Measurement"] = Relationship(back_populates="sensor")


class Measurement(SQLModel, table=True):
    __tablename__ = "measurement"
    id: int | None = Field(default=None, primary_key=True)
    sensor_id: int = Field(foreign_key="sensor.id")
    value: float
    unit: str
    measured_at: str
    sensor: Sensor | None = Relationship(back_populates="measurements")
```

Lalu buka **schemas.py**. Schema membatasi field yang boleh diterima dan dikirim API. Untuk latihan ini, PUT mengganti seluruh field sehingga schema update memakai field yang sama dengan schema create.

```
from sqlmodel import SQLModel


class SensorCreate(SQLModel):
    code: str
    location: str
    parameter: str


class SensorUpdate(SensorCreate):
    pass


class SensorRead(SensorCreate):
    id: int


class MeasurementCreate(SQLModel):
    sensor_id: int
    value: float
    unit: str
    measured_at: str


class MeasurementUpdate(MeasurementCreate):
    pass


class MeasurementRead(MeasurementCreate):
    id: int
```

Pemisahan model dan schema
Model ORM menjelaskan tabel dan relasi. Schema API menjelaskan bentuk JSON. Pemisahan ini membuat API dapat mengatur data masuk/keluar tanpa mencampur route dengan struktur tabel.

5. Langkah 2 - Koneksi Database dan Entry Point

Di **database.py**, buat engine SQLite dan dependency sesi. Satu sesi dibuka untuk request, lalu ditutup setelah request selesai.

```
from sqlmodel import Session, SQLModel, create_engine


DATABASE_URL = "sqlite:///lab_sensor.db"
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
    echo=True,
)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session
```

Di **main.py**, import router dan pasang ke aplikasi FastAPI. Router mengimpor CRUD dan model lebih dulu, sehingga metadata tabel sudah terdaftar saat create_db_and_tables dipanggil.

```
from fastapi import FastAPI

from database import create_db_and_tables
from routers.measurements import router as measurements_router
from routers.sensors import router as sensors_router


create_db_and_tables()
app = FastAPI(title="API Pengukuran Sensor Laboratorium")
app.include_router(sensors_router)
app.include_router(measurements_router)


@app.get("/")
def home():
    return {"message": "API pengukuran sensor siap digunakan", "docs": "/docs"}
```

Alur saat request
main.py memasang router. Router memanggil fungsi CRUD. Fungsi CRUD memakai sesi dari database.py, lalu SQLModel menerjemahkan objek Python menjadi operasi SQLite.

6. Langkah 3 - Fungsi CRUD di crud.py

Semua fungsi database ditempatkan di **crud.py**. Potongan berikut menunjukkan operasi untuk Sensor. Fungsi menerima Session dan schema, lalu mengembalikan objek model atau hasil query.

```
from sqlmodel import Session, select

from models import Measurement, Sensor
from schemas import MeasurementCreate, MeasurementUpdate, SensorCreate, SensorUpdate


def create_sensor(session: Session, data: SensorCreate) -> Sensor:
    sensor = Sensor.model_validate(data)
    session.add(sensor)
    session.commit()
    session.refresh(sensor)
    return sensor


def get_sensors(session: Session) -> list[Sensor]:
    statement = select(Sensor).order_by(Sensor.id)
    return session.exec(statement).all()


def get_sensor(session: Session, sensor_id: int) -> Sensor | None:
    return session.get(Sensor, sensor_id)


def update_sensor(session: Session, sensor: Sensor, data: SensorUpdate) -> Sensor:
    for key, value in data.model_dump().items():
        setattr(sensor, key, value)
    session.add(sensor)
    session.commit()
    session.refresh(sensor)
    return sensor


def sensor_has_measurements(session: Session, sensor_id: int) -> bool:
    statement = select(Measurement.id).where(Measurement.sensor_id == sensor_id)
    return session.exec(statement).first() is not None


def delete_sensor(session: Session, sensor: Sensor) -> None:
    session.delete(sensor)
    session.commit()
```

Mengapa delete_sensor tidak menghapus pengukuran?
Aplikasi menjaga aturan relasi: pengukuran dihapus lebih dahulu. Jika masih ada pengukuran terkait, router mengembalikan status 409 Conflict.

7. Langkah 4 - CRUD Pengukuran di crud.py

Tambahkan fungsi berikut di file **crud.py** yang sama. Query get_measurements_for_sensor memperlihatkan cara memilih baris berdasarkan foreign key.

```
def create_measurement(session: Session, data: MeasurementCreate) -> Measurement:
    measurement = Measurement.model_validate(data)
    session.add(measurement)
    session.commit()
    session.refresh(measurement)
    return measurement


def get_measurements(session: Session) -> list[Measurement]:
    statement = select(Measurement).order_by(Measurement.id)
    return session.exec(statement).all()


def get_measurement(
    session: Session, measurement_id: int
) -> Measurement | None:
    return session.get(Measurement, measurement_id)


def get_measurements_for_sensor(
    session: Session, sensor_id: int
) -> list[Measurement]:
    statement = (
        select(Measurement)
        .where(Measurement.sensor_id == sensor_id)
        .order_by(Measurement.id)
    )
    return session.exec(statement).all()


def update_measurement(
    session: Session, measurement: Measurement, data: MeasurementUpdate
) -> Measurement:
    for key, value in data.model_dump().items():
        setattr(measurement, key, value)
    session.add(measurement)
    session.commit()
    session.refresh(measurement)
    return measurement


def delete_measurement(session: Session, measurement: Measurement) -> None:
    session.delete(measurement)
    session.commit()
```

Fungsi CRUD tidak membentuk respons HTTP. Pemeriksaan sensor tidak ditemukan dan pemilihan status HTTP dilakukan di router, sehingga lapisan akses data tetap mudah digunakan kembali.

8. Langkah 5 - Endpoint Sensor di routers/sensors.py

Router menerima request, memvalidasi resource, lalu memanggil CRUD. Prefix pada APIRouter membuat semua endpoint di file ini diawali /sensors.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlmodel import Session

import crud
from database import get_session
from schemas import MeasurementRead, SensorCreate, SensorRead, SensorUpdate


router = APIRouter(prefix="/sensors", tags=["Sensors"])


@router.post("/", response_model=SensorRead, status_code=201)
def create_sensor(data: SensorCreate, session: Session = Depends(get_session)):
    return crud.create_sensor(session, data)


@router.get("/", response_model=list[SensorRead])
def read_sensors(session: Session = Depends(get_session)):
    return crud.get_sensors(session)


@router.get("/{sensor_id}/measurements", response_model=list[MeasurementRead])
def read_measurements_for_sensor(
    sensor_id: int, session: Session = Depends(get_session)
):
    sensor = crud.get_sensor(session, sensor_id)
    if sensor is None:
        raise HTTPException(status_code=404, detail="Sensor tidak ditemukan")
    return crud.get_measurements_for_sensor(session, sensor_id)


@router.get("/{sensor_id}", response_model=SensorRead)
def read_sensor(sensor_id: int, session: Session = Depends(get_session)):
    sensor = crud.get_sensor(session, sensor_id)
    if sensor is None:
        raise HTTPException(status_code=404, detail="Sensor tidak ditemukan")
    return sensor
```

Endpoint PUT memakai pola yang sama: ambil sensor, kembalikan 404 bila tidak ada, lalu panggil **crud.update_sensor(session, sensor, data)**. Endpoint DELETE memeriksa sensor dan **crud.sensor_has_measurements**, mengembalikan 409 bila masih ada catatan anak, lalu memanggil **crud.delete_sensor**.

9. Langkah 6 - Endpoint Pengukuran

Simpan endpoint pengukuran pada **routers/measurements.py**. Router memeriksa bahwa sensor_id yang dikirim memang ada sebelum membuat atau memindahkan pengukuran.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlmodel import Session

import crud
from database import get_session
from schemas import MeasurementCreate, MeasurementRead, MeasurementUpdate


router = APIRouter(prefix="/measurements", tags=["Measurements"])


@router.post("/", response_model=MeasurementRead, status_code=201)
def create_measurement(
    data: MeasurementCreate,
    session: Session = Depends(get_session),
):
    sensor = crud.get_sensor(session, data.sensor_id)
    if sensor is None:
        raise HTTPException(status_code=404, detail="Sensor tidak ditemukan")
    return crud.create_measurement(session, data)


@router.get("/", response_model=list[MeasurementRead])
def read_measurements(session: Session = Depends(get_session)):
    return crud.get_measurements(session)


@router.get("/{measurement_id}", response_model=MeasurementRead)
def read_measurement(
    measurement_id: int,
    session: Session = Depends(get_session),
):
    measurement = crud.get_measurement(session, measurement_id)
    if measurement is None:
        raise HTTPException(status_code=404, detail="Pengukuran tidak ditemukan")
    return measurement


@router.delete("/{measurement_id}")
def delete_measurement(
    measurement_id: int,
    session: Session = Depends(get_session),
):
    measurement = crud.get_measurement(session, measurement_id)
    if measurement is None:
        raise HTTPException(status_code=404, detail="Pengukuran tidak ditemukan")
    crud.delete_measurement(session, measurement)
    return {"ok": True}
```

Endpoint PUT memakai alur yang sama: cari pengukuran, cari sensor baru dari data.sensor_id, lalu panggil **crud.update_measurement(session, measurement, data)**. Semua endpoint terlihat di /docs dengan kelompok Sensors dan Measurements.

Catatan

Isi berkas lengkap
Contoh di PDF menampilkan bagian penting dan pola tiap lapisan. File proyek yang disertakan memuat implementasi lengkap semua endpoint GET, POST, PUT, dan DELETE.

10. Langkah 7 - Menjalankan dan Membuka API

Pastikan terminal berada di folder yang berisi main.py. Jalankan server dengan perintah berikut:

```
.\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

Jika berhasil, terminal menampilkan alamat server lokal. Biarkan terminal tetap berjalan, lalu buka alamat berikut di browser:

```
http://127.0.0.1:8000/docs
```

Klik endpoint, pilih **Try it out**, isi JSON, lalu tekan **Execute**. FastAPI menampilkan URL, status response, dan JSON hasil. Tidak perlu membuat halaman web terlebih dahulu.

Uji pertama: tambah sensor

Buka POST /sensors/, lalu kirim body berikut. Nilai id pada respons dibuat oleh SQLite; gunakan id tersebut pada pengujian pengukuran.

```
{
  "code": "S-01",
  "location": "Panel Praktikum A",
  "parameter": "tegangan"
}
```

Hasil yang diharapkan: status 201 Created dan respons JSON sensor dengan id, code, location, serta parameter. Kemudian jalankan GET /sensors/ dan pastikan sensor muncul di daftar.

Berkas database
Setelah server dijalankan, file **lab_sensor.db** muncul di folder proyek. Jangan menghapusnya selama masih ingin mempertahankan data praktikum.

11. Langkah 8 - Menguji CRUD dan Relasi

Lakukan pengujian berurutan. Sesuaikan angka id dengan respons yang diterima dari API, karena id dapat berbeda di setiap komputer.

Urutan	Request di /docs	Body / hasil yang diharapkan
1. Create sensor	POST /sensors/	Gunakan JSON S-01 pada halaman sebelumnya. Catat id sensor.
2. Read sensor	GET /sensors/ dan GET /sensors/{id}	Sensor baru terlihat dalam daftar dan detail.
3. Create reading	POST /measurements/	Kirim JSON pengukuran di bawah. Gunakan id sensor yang benar.
4. Read relation	GET /sensors/{id}/measurements	Daftar hanya berisi pengukuran milik sensor tersebut.
5. Update reading	PUT /measurements/{id}	Ganti value, misalnya dari 220.5 ke 221.0.
6. Delete reading	DELETE /measurements/{id}	Respons ok true. GET daftar tidak lagi menampilkan catatan.
7. Update sensor	PUT /sensors/{id}	Kirim seluruh field, misalnya location baru.
8. Delete sensor	DELETE /sensors/{id}	Berhasil setelah pengukuran terkait dihapus.

Contoh body untuk POST atau PUT pengukuran

```
{
  "sensor_id": 1,
  "value": 220.5,
  "unit": "V",
  "measured_at": "2026-10-21T09:30:00"
}
```

Contoh update sensor

```
{
  "code": "S-01",
  "location": "Panel Praktikum B",
  "parameter": "tegangan"
}
```

Urutan penghapusan
Hapus pengukuran lebih dahulu, baru sensor. API sengaja menolak penghapusan sensor yang masih memiliki pengukuran dengan status 409 Conflict.

12. Membaca Respons dan Menemukan Kesalahan

HTTP status dan pesan respons membantu membedakan masalah validasi, data yang tidak ditemukan, serta aturan relasi.

Kondisi uji	Respons yang diharapkan	Makna
Request sensor valid	201 Created	Objek baru berhasil dibuat.
Mencari id yang tidak ada	404 Not Found	Resource tidak ditemukan.
sensor_id pengukuran tidak ada	404 Not Found	Pengukuran tidak boleh menunjuk sensor yang tidak ada.
value dikirim sebagai teks non-angka	422 Unprocessable Entity	Validasi gagal karena nilai tidak dapat dibaca sebagai angka.
Menghapus sensor yang masih punya pengukuran	409 Conflict	Aturan relasi meminta catatan anak dihapus lebih dahulu.

Amati juga terminal server

Karena engine dibuat dengan echo=True, SQL yang dihasilkan ORM ditampilkan di terminal. Cari perintah CREATE TABLE, INSERT, SELECT, UPDATE, dan DELETE. Hubungkan setiap operasi SQL dengan request HTTP yang dikirim.

Pemeriksaan persistensi

- Buat sensor dan satu pengukuran.
- Hentikan server dengan Ctrl+C.
- Jalankan lagi perintah Uvicorn, lalu buka GET /sensors/.
- Pastikan data masih ada. Jika tetap ada, data tersimpan di file SQLite, bukan hanya di memori program.

Jika ingin mulai dari database kosong
Hentikan server terlebih dahulu, hapus file **lab_sensor.db**, lalu jalankan kembali aplikasi. Tabel akan dibuat ulang saat main.py dimuat.

Catatan: menghapus database menghilangkan seluruh data. Simpan salinan file jika data masih diperlukan.

13. Tugas, Penilaian, dan Pengembangan

Tugas mahasiswa

- Ganti data contoh dengan sedikitnya dua sensor: tegangan dan arus atau suhu.
- Masukkan sedikitnya tiga pengukuran untuk salah satu sensor.
- Buktikan GET untuk satu sensor hanya menampilkan pengukuran yang terkait.
- Tunjukkan satu kasus 404, satu kasus 422 dengan value non-angka, dan penolakan 409 saat menghapus sensor yang masih memiliki pengukuran.
- Lampirkan diagram relasi, tangkapan layar pengujian /docs, dan catatan singkat tentang file SQLite.

Rubrik ringkas

Aspek	Bobot	Indikator
Desain relasi	25%	PK, FK, dan kardinalitas satu-ke-banyak benar.
Pemetaan ORM	25%	Model dan tabel sesuai kamus data; perubahan tersimpan.
CRUD dan REST	30%	GET, POST, PUT, DELETE bekerja untuk kedua entitas.
Validasi dan bukti	20%	Respons error benar dan bukti pengujian dapat dibaca.

Pengembangan opsional

Tambahkan endpoint untuk menghitung rata-rata atau nilai maksimum dari seluruh pengukuran sensor. Setelah mahasiswa memahami CRUD, materi konkurensi dapat diberikan pada praktikum terpisah: pemanggilan beberapa API secara paralel, timeout/fallback, dan simulasi lost update.

Troubleshooting cepat

Pesan atau gejala	Langkah perbaikan
No module named fastapi/sqlmodel	Pasang paket dengan .venv\Scripts\python.exe -m pip install -r requirements.txt.
Address already in use	Hentikan server lama dengan Ctrl+C atau jalankan pada port lain dengan --port 8001.
404 untuk pengukuran	Pastikan sensor_id pada body sama dengan id yang dikembalikan POST /sensors/.
File database tidak terlihat	Periksa folder kerja terminal; SQLite membuat lab_sensor.db relatif terhadap folder itu.

Sumber belajar

FastAPI, tutorial basis data SQL: fastapi.tiangolo.com/tutorial/sql-databases/
FastAPI, dokumentasi interaktif: fastapi.tiangolo.com/tutorial/first-steps/
SQLAlchemy, ORM Quick Start: docs.sqlalchemy.org/en/21/orm/quickstart.html
Bahan kuliah: Pengenalan RESTful API dan Format JSON; Konkurensi dan Penanganan Kegagalan.

Selesai
Mahasiswa telah membangun API yang menyimpan objek Python ke basis data relasional, menyediakan CRUD, dan menjaga keterkaitan pengukuran dengan sensor.